

INTERVIEW PREPARATION GUIDE

OOPs Interview Questions & Answers

75 Object-Oriented Programming questions (C++) compiled by cross-referencing multiple interview-prep sources, covering everything from the four pillars to C++-specific gotchas like virtual destructors, object slicing, and the diamond problem.

Sources cross-referenced: GeeksforGeeks (OOP & C++ OOP tracks), InterviewBit, Simplilearn, Edureka, MyGreatLearning, ScienceTechEasy, FinalRoundAI. Questions appearing on 3+ sources are marked **Frequently Asked**; C++-only conceptual traps are marked **C++ Specific**.

COVERAGE

Fundamentals → Advanced

LANGUAGE

C++

FORMAT

10 Topic Sections

Table of Contents

01	OOP Fundamentals & Core Pillars	10 questions
02	Classes & Objects	7 questions
03	Constructors & Destructors	9 questions
04	Inheritance	9 questions
05	Polymorphism Deep-Dive	9 questions
06	Access Modifiers, Friend & Static	6 questions
07	Abstract Classes & Interfaces	5 questions
08	Operator Overloading	5 questions
09	Memory & Object Lifetime	6 questions
10	Advanced / Tricky Questions	9 questions

01 OOP Fundamentals & Core Pillars

Q1. What is Object-Oriented Programming (OOP)?

FREQUENTLY ASKED

OOP is a programming paradigm that organizes code around **objects** — bundles of data (attributes) and behavior (methods) — rather than around a sequence of functions acting on separate data. It's built on four pillars: encapsulation, abstraction, inheritance, and polymorphism, aimed at modeling real-world entities and making large codebases easier to reason about, extend, and maintain.

Q2. How does OOP differ from procedural programming?

FREQUENTLY ASKED

	Procedural Programming	Object-Oriented Programming
Organized around	Functions operating on data	Objects bundling data + behavior
Data access	Usually global/shared	Encapsulated within objects
Reuse mechanism	Function calls	Inheritance & composition
Change resilience	Harder to extend without touching existing code	Easier to extend via new classes/overrides

Q3. What are the four main pillars of OOP?

FREQUENTLY ASKED

- **Encapsulation** — bundling data and the methods that operate on it, hiding internal state behind a controlled interface.
- **Abstraction** — exposing only essential behavior while hiding implementation complexity.
- **Inheritance** — deriving new classes from existing ones to reuse and extend behavior.
- **Polymorphism** — letting different types respond to the same interface/call in type-specific ways.

Q4. What is Encapsulation? Give an example.

FREQUENTLY ASKED

Encapsulation is keeping an object's internal data private and exposing controlled access through public methods (getters/setters), so the object's invariants can't be violated by external code reaching in directly.

```
class BankAccount {
private:
    double balance;
public:
    BankAccount(double initial) : balance(initial) {}
    void deposit(double amt) { if (amt > 0) balance += amt; }
    double getBalance() const { return balance; }
};
```

Here `balance` can't be set to an invalid value directly from outside — all changes go through `deposit()`, which can enforce rules.

Q5. What is Abstraction, and how does it differ from Encapsulation?

FREQUENTLY ASKED

Abstraction is about hiding *complexity* — exposing only the "what" (a simple interface) while hiding the "how" (implementation details), e.g. calling `car.start()` without knowing the ignition sequence.

Encapsulation is about hiding *data* — restricting direct access to an object's internal state. They're complementary: encapsulation is the mechanism (access control) that's often used to achieve abstraction (a simplified interface). In C++, abstraction is typically achieved with abstract classes/pure virtual functions.

Q6. What is Inheritance, and what is its purpose?

FREQUENTLY ASKED

Inheritance lets a class (the **derived**/child class) acquire the properties and behavior of another class (the **base**/parent class), modeling an "is-a" relationship. Its purpose is code reuse and establishing a type hierarchy so common behavior lives in one place and specialized behavior is added or overridden in derived classes.

```
class Animal {
public:
    void eat() { cout << "eating" << endl; }
};
class Dog : public Animal {
public:
    void bark() { cout << "barking" << endl; }
};
// Dog d; d.eat(); d.bark(); - Dog inherits eat() for free
```

Q7. What is Polymorphism, and what are its types?

FREQUENTLY ASKED

Polymorphism ("many forms") lets the same interface/call behave differently depending on the object or arguments involved. C++ supports:

- **Compile-time (static) polymorphism** — function overloading and operator overloading, resolved by the compiler.
- **Runtime (dynamic) polymorphism** — virtual functions and function overriding, resolved during execution based on the object's actual type.

Q8. What are the advantages of OOP?

FREQUENTLY ASKED

- **Modularity** — code is organized into self-contained, reusable classes.
- **Reusability** — inheritance and composition let existing code be extended rather than rewritten.
- **Maintainability** — encapsulation limits how changes ripple across the codebase.
- **Flexibility** — polymorphism lets new types plug into existing code paths without modifying them.
- Naturally models real-world entities, making designs more intuitive to reason about.

Q9. What are the limitations or disadvantages of OOP?

- Can introduce unnecessary complexity/overhead for genuinely simple, linear scripts.
- Deep inheritance hierarchies become hard to trace and are fragile to change ("fragile base class" problem).
- Objects and virtual dispatch carry a small runtime/memory overhead compared to plain procedural code.
- Overuse of inheritance where composition would be simpler is a common design mistake.

Q10. Is C++ a "purely" object-oriented language?

No — C++ is a **multi-paradigm** language: it fully supports OOP but also allows plain procedural code (free functions, global variables) outside any class, unlike languages such as Java where essentially everything must live inside a class. This flexibility is intentional, letting C++ be used for systems programming where OOP overhead isn't always wanted.

02 Classes & Objects

Q11. What is a class?

FREQUENTLY ASKED

A class is a user-defined **blueprint** or template that defines the data members (attributes) and member functions (behavior) that its instances will have. No memory is allocated for the data members themselves just by defining a class — memory is allocated only when an object is created.

Q12. What is an object?

FREQUENTLY ASKED

An object is a concrete **instance** of a class — it has actual, allocated storage for the class's data members and can invoke the class's methods. Many distinct objects can be created from the same class, each with its own independent state.

```
class Car { public: string model; };  
Car c1; c1.model = "Sedan"; // c1 is an object (instance) of Car  
Car c2; c2.model = "SUV";   // c2 is a separate, independent object
```

Q13. What is the difference between a class and an object?

FREQUENTLY ASKED

	Class	Object
Nature	Logical blueprint/definition	Physical, allocated instance
Memory	No memory for data until instantiated	Occupies real memory
Count	Declared once	Many objects can exist per class

Q14. What is the difference between `struct` and `class` in C++?

FREQUENTLY ASKED

C++ SPECIFIC

Functionally almost identical in C++ (unlike C, where `struct` can't have methods) — both can have constructors, member functions, access specifiers, and support inheritance. The only real differences are the **defaults**: `struct` members and base classes are `public` by default, while `class` members and base classes are `private` by default. By convention, `struct` is used for simple, passive data-holding types, and `class` for types with significant behavior/invariants.

Q15. Is memory allocated when you just define a class?

FREQUENTLY ASKED

No. A class definition only describes the layout and behavior — it's a compile-time construct. Memory is allocated only when an object of that class is actually created (on the stack, heap, or as a global/static), at which point space is reserved for each non-static data member.

Q16. What is the `this` pointer?

An implicit pointer available inside every non-static member function that points to the object the method was called on. It's used to disambiguate a member variable from a same-named parameter, to return `*this` for method chaining, and internally by the compiler to pass "which object" into every non-static call.

```
class Point {
    int x;
public:
    Point& setX(int x) { this->x = x; return *this; } // enables chaining
};
```

Q17. What is composition, and how does it differ from simple class usage?

Composition means a class holds an object of another class as a member, modeling a "has-a" relationship (as opposed to inheritance's "is-a"). The contained object's lifetime is tied to the containing object's lifetime.

```
class Engine { public: void start() { /* ... */ } };
class Car {
    Engine engine; // Car "has-a" Engine
public:
    void start() { engine.start(); }
};
```

03 Constructors & Destructors

Q18. What is a constructor?

FREQUENTLY ASKED

A special member function, named the same as the class, automatically invoked when an object is created — used to initialize the object's data members. It has no return type (not even `void`) and can be overloaded to support different ways of initializing an object.

Q19. What are the types of constructors in C++?

FREQUENTLY ASKED

- **Default constructor** — takes no arguments; if you define no constructors at all, the compiler generates one implicitly.
- **Parameterized constructor** — takes arguments to initialize members with specific values.
- **Copy constructor** — initializes a new object as a copy of an existing object of the same class.

```
class Point {
    int x, y;
public:
    Point() : x(0), y(0) {}           // default
    Point(int x, int y) : x(x), y(y) {} // parameterized
    Point(const Point& other) : x(other.x), y(other.y) {} // copy
};
```

Q20. What is a copy constructor, and when is it invoked?

FREQUENTLY ASKED

A constructor of the form `ClassName(const ClassName& other)` that creates a new object as a copy of an existing one. It's invoked when: an object is initialized from another (`Point p2 = p1;` or `Point p2(p1);`), an object is passed by value to a function, or an object is returned by value (in the absence of move semantics/RVO).

Q21. Copy constructor vs assignment operator (`operator=`) — what's the difference?

FREQUENTLY ASKED

C++ SPECIFIC

The **copy constructor** creates a brand-new object from an existing one (no destination object exists yet). The **assignment operator** copies values into an *already-existing* object, which may need to first release/clean up resources it currently holds.

```
Point p1(1, 2);
Point p2 = p1;   // copy constructor (p2 doesn't exist yet)
Point p3;
p3 = p1;         // assignment operator (p3 already exists)
```

Q22. Can constructors be overloaded?

FREQUENTLY ASKED

Yes — a class can define multiple constructors as long as their parameter lists differ in number or type, giving callers different ways to initialize an object (as shown in the default/parameterized/copy example above). The compiler picks the matching overload based on the arguments supplied at the call site.

Q23. What is constructor chaining / delegating constructors?

Since C++11, a constructor can call another constructor of the *same* class in its member-initializer list to avoid duplicating initialization logic ("delegating constructors"). This is distinct from a derived class's constructor implicitly invoking its base class's constructor, which also happens automatically before the derived constructor's body runs.

```
class Point {
    int x, y;
public:
    Point(int x, int y) : x(x), y(y) {}
    Point() : Point(0, 0) {} // delegates to the parameterized constructor
};
```

Q24. What is a destructor?

FREQUENTLY ASKED

A special member function named `~ClassName()`, automatically called when an object goes out of scope or is explicitly `deleted`, used to release resources the object acquired (dynamic memory, file handles, locks). It takes no arguments and cannot be overloaded — a class has exactly one destructor.

Q25. Can a destructor be overloaded, or can a class have multiple destructors?

No. A destructor takes no parameters and cannot be overloaded — a class can only ever have one destructor, since the compiler needs a single, unambiguous way to clean up an object regardless of how it was constructed.

Q26. Why should a base class destructor be declared virtual ?

FREQUENTLY ASKED

C++ SPECIFIC

If a base class destructor is **not** virtual and a derived object is deleted through a `Base*` pointer, only the base class's destructor runs — the derived class's destructor (and cleanup of any resources it owns) is skipped, causing undefined behavior and resource/memory leaks. Marking the base destructor `virtual` ensures the correct, most-derived destructor runs first, followed by each base destructor up the chain.

```
class Base {
public:
    virtual ~Base() { cout << "Base cleanup\n"; }
};
class Derived : public Base {
public:
    ~Derived() { cout << "Derived cleanup\n"; }
};
Base* b = new Derived();
delete b; // with virtual ~Base(): prints both; without it: only "Base cleanup"
```

Q27. What are the types of inheritance in C++?

FREQUENTLY ASKED

- **Single** — one derived class, one base class.
- **Multiple** — one derived class inherits from two or more base classes.
- **Multilevel** — a chain: C derives from B, which derives from A.
- **Hierarchical** — multiple derived classes share a single base class.
- **Hybrid** — a combination of the above (e.g. hierarchical + multiple), which is where the diamond problem can arise.

Q28. What's the difference between multiple and multilevel inheritance?

Multiple inheritance: one class directly inherits from two or more unrelated base classes at the same level (class C : public A, public B). **Multilevel inheritance:** a linear chain of inheritance across generations (class C : public B, where class B : public A) — C indirectly inherits from A through B.

Q29. What are the limitations/drawbacks of inheritance?

FREQUENTLY ASKED

- Tight coupling between base and derived classes — changes to the base class can unexpectedly break derived classes ("fragile base class" problem).
- Deep hierarchies are hard to understand, debug, and refactor.
- Multiple inheritance can introduce ambiguity (the diamond problem).
- Inheriting for code reuse alone, without a genuine "is-a" relationship, leads to awkward, misleading designs — composition is often a better fit.

Q30. What is the Diamond Problem, and how does C++ resolve it?

FREQUENTLY ASKED

C++ SPECIFIC

When class D inherits from both B and C, and both B and C inherit from a common base A, D ends up with **two separate copies** of A's members — referencing a member of A through D becomes ambiguous (which copy?). C++ resolves this with **virtual inheritance**: declaring B and C's inheritance from A as virtual ensures only a single, shared copy of A exists in D.

```
class A { public: int x; };  
class B : virtual public A {};  
class C : virtual public A {};  
class D : public B, public C {}; // D has exactly one A::x now
```

Q31. What is virtual inheritance?

C++ SPECIFIC

Declaring a base class as `virtual` in an inheritance list tells the compiler that this base should be shared across all paths that inherit it virtually, rather than duplicated — the mechanism that solves the diamond problem (see above). It comes with a small runtime cost (an extra pointer for locating the shared base subobject) and is used specifically when hybrid/multiple inheritance could otherwise duplicate a common ancestor.

Q32. What is the difference between Association, Aggregation, and Composition?

FREQUENTLY ASKED

	Association	Aggregation	Composition
Relationship	General "uses-a" link	Weak "has-a"	Strong "has-a" (owns)
Lifetime dependency	Independent	Part can outlive the whole	Part dies with the whole
Example	A Teacher and a Student	A Department and its Employees	A House and its Rooms

Q33. What's the difference between "Is-A" and "Has-A" relationships?

"Is-A" is modeled with **inheritance** — a Dog is-a Animal, so Dog derives from Animal. "Has-A" is modeled with **composition/aggregation** — a Car has-a Engine, so Car holds an Engine member rather than inheriting from it. Misusing inheritance for a "has-a" relationship is a very common design mistake flagged in interviews.

Q34. Why is "favor composition over inheritance" a common recommendation?

Inheritance creates tight coupling: a derived class depends on its base class's implementation details, and changes to the base can silently break every subclass. Composition keeps classes more loosely coupled and flexible — behavior can be swapped at runtime by holding a different composed object, whereas an inheritance hierarchy is fixed at compile time. Composition is generally preferred unless there's a genuine, stable "is-a" relationship with shared polymorphic behavior.

Q35. Can a derived class access the private members of its base class?

No — private members are accessible only within the class that declares them, not even by derived classes. If a base class wants to expose a member to derived classes (but not to outside code), it should be declared `protected` instead of `private`.

05 Polymorphism Deep-Dive

Q36. Function Overloading vs Function Overriding — what's the difference?

FREQUENTLY ASKED

	Overloading	Overriding
Where	Same class (or scope)	Base and derived class
Signature	Must differ (params/types)	Must match exactly
Binding	Compile-time (static)	Runtime (dynamic), needs <code>virtual</code>
Purpose	Multiple ways to call similarly-named operations	Specialize inherited behavior

Q37. Compile-time vs Runtime polymorphism — explain the difference.

FREQUENTLY ASKED

Compile-time (static) polymorphism is resolved by the compiler based on the static types/argument list at compile time — function overloading, operator overloading, templates. **Runtime (dynamic) polymorphism** is resolved during program execution based on the object's actual (dynamic) type — achieved via virtual functions and inheritance, using the vtable mechanism.

Q38. What is a virtual function?

FREQUENTLY ASKED

A member function declared with the `virtual` keyword in a base class that a derived class can override; calling it through a base class pointer/reference invokes the derived class's version at runtime, based on the object's actual type — the core mechanism behind runtime polymorphism.

```
class Shape {
public:
    virtual double area() const { return 0; }
};
class Circle : public Shape {
    double r;
public:
    Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
};
Shape* s = new Circle(2);
s->area(); // calls Circle::area() even though s is typed as Shape*
```

Q39. What is a pure virtual function, and what does it make the class?

FREQUENTLY ASKED

A virtual function declared with `= 0` and no implementation in the base class, e.g. `virtual double area() const = 0;`. It forces every concrete derived class to provide its own implementation. A class containing at least one pure virtual function becomes an **abstract class** and cannot be instantiated directly.

Q40. Virtual function vs pure virtual function — what's the difference?

FREQUENTLY ASKED

A **virtual function** has a (possibly default) implementation in the base class, which derived classes *may* override. A **pure virtual function** has no implementation in the base class (= 0) and derived classes *must* override it to be instantiable. A class with only ordinary virtual functions can still be instantiated; a class with any pure virtual function cannot.

Q41. What is the difference between early binding and late binding (static vs dynamic binding)?

C++ SPECIFIC

Early (static) binding: the compiler determines which function to call at compile time — used for non-virtual functions, overloaded functions, and static/private methods. **Late (dynamic) binding:** the actual function to call is determined at runtime through the vtable, based on the object's real type — used for virtual function calls made through a base class pointer or reference.

Q42. What is function/name hiding in C++ inheritance?

C++ SPECIFIC

If a derived class defines a function with the same *name* as one in the base class (even with a different signature), it hides **all** overloads of that name from the base class for objects accessed through the derived type — not just the one it "overloads." This surprises many developers expecting normal overload resolution across the hierarchy. A `using Base::functionName;` declaration in the derived class brings the base overloads back into scope.

```
class Base {
public:
    void greet() { cout << "Hi"; }
    void greet(int x) { cout << "Hi " << x; }
};
class Derived : public Base {
public:
    void greet(string s) { cout << "Hello " << s; } // hides BOTH Base::greet overloads
};
Derived d;
d.greet("Sam"); // OK
// d.greet();   // ERROR – Base::greet() is hidden, not overloaded
d.Base::greet(); // still reachable explicitly
```

Q43. Can static or private member functions be "overridden"?

No, not in the polymorphic sense. **Static** methods belong to the class itself, not to any object, so they're resolved at compile time based on the static type — a derived class can define a same-named static method, but that's hiding, not overriding (no dynamic dispatch). **Private** virtual functions technically *can* be overridden by a derived class (overriding only requires visibility at the point of declaration, not the call site) — a subtle, less commonly known fact often used as a trick interview question.

Q44. What happens if you call a virtual function from a constructor or destructor?

It does **not** resolve to an override in a more-derived class. While a base class's constructor is running, the object is still considered to be of the base type (the derived part hasn't been constructed yet), so a virtual call inside it invokes the base class's own version — never a derived override. The same applies in reverse during destruction. This is a common C++ gotcha and a frequent trick question.

06 Access Modifiers, Friend & Static

Q45. What are access specifiers in C++?

FREQUENTLY ASKED

- **public** — accessible from anywhere the object is visible.
- **private** — accessible only within the class itself (and its friends).
- **protected** — accessible within the class and any classes derived from it, but not from outside code.

Q46. What's the difference between private and protected ?

FREQUENTLY ASKED

Both block access from outside the class, but protected members remain accessible to **derived classes**, while private members are visible only within the exact class that declares them — not even subclasses can touch them directly. Use protected when subclasses legitimately need direct access to internal state; otherwise prefer private with accessor methods for tighter encapsulation.

Q47. What are friend functions and friend classes?

FREQUENTLY ASKED

A friend declaration grants a specific external function or an entire external class access to a class's private and protected members, bypassing normal encapsulation for that specific relationship. It's used sparingly — e.g. for operator overloads (like `operator<<` for printing) that need internal access but can't naturally be a member function.

```
class Box {
    int width;
    friend void printWidth(const Box& b); // has access despite being non-member
public:
    Box(int w) : width(w) {}
};
void printWidth(const Box& b) { cout << b.width; }
```

Q48. What are static data members and static member functions?

FREQUENTLY ASKED

A static data member is shared by **all** objects of a class (one copy total, not one per object) — useful for counters or shared configuration. A static member function can access only static members (no `this` pointer, since it isn't tied to any specific instance) and can be called via the class name without needing an object.

```
class Counter {
    static int count;
public:
    Counter() { count++; }
    static int getCount() { return count; }
};
int Counter::count = 0; // must be defined outside the class
// Counter::getCount() – callable without any Counter object
```

Q49. What is the scope resolution operator (::)?

It qualifies a name with the scope it belongs to — used to define a class's member function outside the class body (`void MyClass::method() { ... }`), to access a static member (`Counter::getCount()`), or to explicitly call a base class's version of a method that's been hidden/overridden (`Base::method()`).

Q50. What storage classes exist in C++?

- **auto** — default local variable storage (automatically deallocated at scope end).
- **static** — persists for the program's lifetime; retains its value across function calls, or is shared across a class's instances.
- **extern** — declares a variable/function defined in another translation unit.
- **mutable** — allows a class member to be modified even inside a `const` member function.

07 Abstract Classes & Interfaces

Q51. What is an abstract class in C++?

FREQUENTLY ASKED

A class that contains at least one **pure virtual function** (`= 0`), and therefore cannot be instantiated on its own. It exists to define a common interface/contract that concrete derived classes must implement, while potentially also providing some shared, non-pure implementation.

Q52. C++ has no native `interface` keyword — how is an interface simulated?

FREQUENTLY ASKED

By defining a class where **every** member function is a pure virtual function and it has no data members — effectively a pure contract with zero implementation, which is exactly what an "interface" means in languages like Java.

```
class Drawable {
public:
    virtual void draw() const = 0;
    virtual ~Drawable() = default; // still needs a virtual destructor
};
```

Q53. Abstract class vs Interface — what's the difference (conceptually)?

FREQUENTLY ASKED

	Abstract Class	Interface (C++ convention)
Implementation	Can mix pure virtual and regular/implemented methods	All methods pure virtual, no implementation
Data members	Can have state	Typically none
Inheritance	Usually single inheritance base	Commonly inherited multiply, like a "capability"

Q54. Can you instantiate an abstract class?

FREQUENTLY ASKED

No — attempting to create an object of a class with any unimplemented pure virtual function is a compile-time error. You can, however, use a base pointer or reference of that abstract type to point to a fully-implemented derived object (`Shape* s = new Circle(2);`), which is exactly how runtime polymorphism is used in practice.

Q55. What is data abstraction, and how is it achieved in C++?

Data abstraction means exposing only the necessary details of an object's data and hiding the rest — users interact with a simplified view. It's achieved in C++ through classes with private/protected data plus a curated public interface, and through abstract classes that expose only method signatures without implementation.

08 Operator Overloading

Q56. What is operator overloading?

FREQUENTLY ASKED

Defining custom behavior for an existing C++ operator (+, ==, <<, etc.) when applied to objects of a user-defined class, so objects can be used with natural, intuitive syntax instead of named methods.

```
class Complex {
    double re, im;
public:
    Complex(double r, double i) : re(r), im(i) {}
    Complex operator+(const Complex& other) const {
        return Complex(re + other.re, im + other.im);
    }
};
Complex c3 = c1 + c2; // calls operator+
```

Q57. Operator Overloading vs Function Overloading — what's the difference?

FREQUENTLY ASKED

Both are forms of compile-time polymorphism resolved by argument types, but **function overloading** defines multiple functions sharing a name, while **operator overloading** specifically redefines what an existing operator symbol does for a custom type, using the special operator *symbol* syntax (e.g. operator+, operator==).

Q58. Which operators cannot be overloaded in C++?

FREQUENTLY ASKED

A small fixed set: the scope resolution operator ::, the member access operator ., the pointer-to-member operator .*, the ternary conditional operator ?:, and the sizeof operator. These are excluded because allowing them to be overloaded would undermine fundamental language guarantees (e.g. sizeof must always reflect true object size).

Q59. What is the Rule of Three / Rule of Five?

C++ SPECIFIC

Rule of Three: if a class needs to define any one of a custom destructor, copy constructor, or copy assignment operator (typically because it manages a raw resource like dynamic memory), it almost certainly needs to define all three — the compiler-generated defaults for the other two will likely do the wrong thing (e.g. shallow-copying a pointer twice). **Rule of Five** (C++11+) extends this to also cover the move constructor and move assignment operator for efficient resource transfer.

Q60. Prefix vs postfix operator overloading (++ / --) — how do they differ in C++?

Both can be overloaded, but they're distinguished by a dummy `int` parameter on the postfix version. Prefix returns a reference to the already-incremented object; postfix must save the old value, increment, and return the old value by copy.

```
class Counter {
    int val = 0;
public:
    Counter& operator++() { ++val; return *this; }           // prefix: ++c
    Counter operator++(int) { Counter tmp = *this; ++val; return tmp; } // postfix: c++
};
```

Q61. Shallow copy vs Deep copy — what's the difference?

FREQUENTLY ASKED

C++ SPECIFIC

A **shallow copy** (the compiler's default copy behavior) copies member values as-is — if a member is a pointer, both the original and the copy end up pointing to the *same* underlying memory, so freeing one affects the other (double-free / dangling pointer risk). A **deep copy** allocates separate memory for the copy and duplicates the pointed-to data, so the two objects are fully independent. Classes owning raw pointers need a custom copy constructor/assignment operator to perform a deep copy.

```
class Buffer {
    int* data;
public:
    Buffer(int size) : data(new int[size]) {}
    // Deep copy constructor — without this, the default shallow copy
    // would leave two Buffers sharing (and double-freeing) the same array.
    Buffer(const Buffer& other) { data = new int[sizeof(*data)]; /* copy contents */ }
    ~Buffer() { delete[] data; }
};
```

Q62. What is the RAII principle?

C++ SPECIFIC

RAII ("Resource Acquisition Is Initialization") ties a resource's lifetime to an object's lifetime: the resource (memory, file handle, mutex lock) is acquired in the constructor and automatically released in the destructor. Since C++ guarantees destructors run when an object goes out of scope (including during stack unwinding from an exception), RAII makes resource cleanup automatic and exception-safe — the foundation behind smart pointers, `std::lock_guard`, and `std::fstream`.

Q63. How do smart pointers help avoid memory leaks?

Smart pointers (`std::unique_ptr`, `std::shared_ptr`, `std::weak_ptr`) wrap a raw pointer in an RAII object that automatically calls `delete` in its destructor. `unique_ptr` enforces sole ownership (non-copyable, only movable); `shared_ptr` uses reference counting so the resource is freed once the last owner is destroyed — eliminating most manual `new/delete` bookkeeping and the leaks/dangling pointers that come from getting it wrong.

Q64. Stack allocation vs heap allocation for objects — what's the trade-off?

Stack objects (`Point p;`) are automatically destroyed when they go out of scope — fast allocation, no manual cleanup, but fixed lifetime tied to scope. **Heap** objects (`Point* p = new Point();`) live until explicitly deleted (or a smart pointer releases them) — more flexible lifetime (can outlive the creating scope, useful for polymorphic objects accessed via base pointers), but slower allocation and the responsibility of manual or smart-pointer-managed cleanup.

Q65. Does C++ have garbage collection? How does that differ from Java?

No — C++ does not have automatic garbage collection built into the language; the programmer (or RAII-based abstractions like smart pointers) is responsible for reclaiming heap memory. Java, by contrast, runs a garbage collector that automatically reclaims objects with no remaining references, at the cost of some runtime overhead and less deterministic cleanup timing (a Java object's `finalize` isn't guaranteed to run promptly, whereas a C++ destructor runs deterministically and immediately).

Q66. What is a dangling pointer, and how does it relate to object destruction?

A dangling pointer points to memory that has already been freed (e.g. after `delete`, or after a stack object's scope ends) — dereferencing it is undefined behavior. Common causes tied to OOP: returning a pointer/reference to a local stack object from a function, or a shallow-copied class whose copy's destructor frees memory the original object still references.

10 Advanced / Tricky Questions

Q67. What is Object Slicing, and how do you prevent it?

C++ SPECIFIC

When a derived-class object is assigned or passed **by value** to a variable/parameter of the base class type, only the base-class portion is copied — the derived-specific members are "sliced off," and any polymorphic behavior is lost from that point on. It's prevented by passing/storing objects through pointers or references instead of by value when polymorphism is needed.

```
class Base { public: virtual void show() { cout << "Base"; } };
class Derived : public Base {
    int extra = 42;
public:
    void show() override { cout << "Derived, extra=" << extra; }
};
Derived d;
Base b = d;          // SLICED - b is just a Base, extra and override are lost
b.show();            // prints "Base"
Base& ref = d;       // NOT sliced - reference preserves the full Derived object
ref.show();          // prints "Derived, extra=42"
```

Q68. How does the vtable/vptr mechanism implement dynamic dispatch?

C++ SPECIFIC

Any class with at least one virtual function gets a compiler-generated **vtable** (virtual table) — a static array of function pointers, one per virtual function, pointing to the correct override for that class. Every object of such a class carries a hidden **vptr** (virtual pointer) set by the constructor to point at its class's vtable. A virtual call through a base pointer follows object's vptr → vtable → correct function slot at runtime, which is how the right override gets called even though the compiler only knows the static (base) type at the call site.

Q69. What is the performance cost of virtual functions?

Two small costs: an extra pointer (vptr) per object (increasing object size), and one extra level of indirection per virtual call (dereferencing the vptr, then indexing into the vtable) compared to a direct function call, which also usually prevents the compiler from inlining the call. In practice this overhead is tiny and rarely the actual bottleneck — it matters mainly in extremely hot, tight loops.

Q70. What are the SOLID principles?

FREQUENTLY ASKED

- **S** — Single Responsibility: a class should have only one reason to change.
- **O** — Open/Closed: open for extension, closed for modification (extend via new code, not by editing existing working code).
- **L** — Liskov Substitution: a derived class object should be usable anywhere a base class object is expected, without breaking correctness.
- **I** — Interface Segregation: prefer several small, specific interfaces over one large, general-purpose one.
- **D** — Dependency Inversion: depend on abstractions (interfaces), not concrete implementations.

Q71. Name a few common design patterns and when you'd use each.

- **Singleton** — ensures a class has exactly one instance with a global access point (e.g. a single configuration manager).
- **Factory** — centralizes object creation logic so callers don't need to know concrete class details.
- **Observer** — lets objects (observers) subscribe to and react to state changes in another object (subject), e.g. event/notification systems.
- **Strategy** — encapsulates interchangeable algorithms behind a common interface, selected at runtime (e.g. different sorting or pricing strategies).

Q72. Why is the Singleton pattern sometimes considered an anti-pattern?

Singletons introduce global mutable state, which makes unit testing harder (tests can't easily substitute a mock instance), creates hidden dependencies between unrelated parts of the code that all reach into the same global instance, and can cause subtle bugs in multi-threaded contexts if initialization/access isn't carefully synchronized. Dependency injection is often preferred as a more testable alternative to enforcing "just one instance."

Q73. What are coupling and cohesion, and why do they matter in OOP design?

Coupling measures how dependent classes are on each other's internals — lower coupling means classes can change independently. **Cohesion** measures how focused a single class's responsibilities are — higher cohesion means a class does one well-defined thing. Good OOP design aims for **low coupling, high cohesion**: classes that are internally focused but only loosely connected to each other, which makes the system easier to change, test, and reason about.

Q74. What is the Curiously Recurring Template Pattern (CRTTP)?

C++ SPECIFIC

A C++ idiom where a class derives from a template base class that is itself parameterized with the derived class: `class Derived : public Base<Derived>`. This lets the base class call methods on the derived class at compile time (via a `static_cast<Derived*>(this)`) without any virtual function overhead — a form of "static polymorphism" used in performance-sensitive code.

```
template <typename T>
class Base {
public:
    void interface() { static_cast<T*>(this)->implementation(); }
};
class Derived : public Base<Derived> {
public:
    void implementation() { cout << "Derived impl, no vtable needed"; }
};
```

Q75. What is const-correctness in C++?

Marking member functions, parameters, and variables `const` wherever they shouldn't modify state, so the compiler enforces those guarantees and catches accidental mutation as a compile error. A `const` member function (`double area() const;`) promises not to modify the object's members, which also allows it to be called on `const` objects and references — important for writing APIs that work correctly with read-only data.